Advanced Migrations

Most Rails developers use the basic facilities of migrations to create and maintain their database schemas. But every now and then it's useful to push migrations just a bit further. This section covers some more advanced migration usage.

Using Native SQL

Migrations give you a database-independent way of maintaining your application's schema. However, if migrations don't contain the methods you need to be able to do what you need to do, you'll need to drop down to database-specific code. Rails provides two ways to do this. One is with `options` arguments to methods like `add_column`. The second is the `execute` method.

When you use `options` or `execute`, you might well be tying your migration to a specific database engine, because any SQL you provide in these two locations uses your database's native syntax.

An example of where you might need to use raw SQL is if you're creating a custom data type inside your database. Postgres, for example, allows you to specify *enumerated types*. Enumerated types work just fine with Rails; but to create them in a migration, you have to use SQL and thus `execute`. Suppose we wanted to create an enumerated type for the various pay types we supported in our checkout form (which we created in Chapter 12, *Task G: Check Out!*):

```
class AddPayTypes < ActiveRecord::Migrations[6.0]
  def up
    execute %{
      CREATE TYPE
        pay_type
      AS ENUM (
        'check',
```